

Strathmore University

@iLabAfrica Centre

Session 11: MLOps II

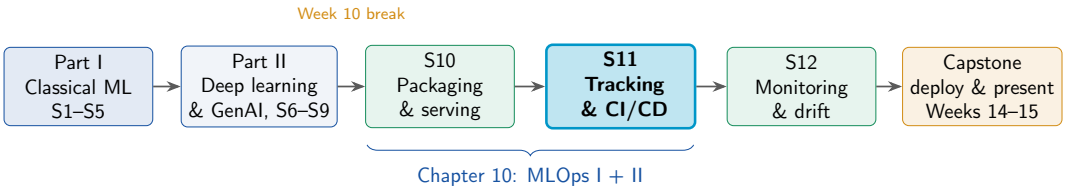
Experiment Tracking & CI/CD

Javier Serrano

Applied Machine Learning

Master in Data Science & Analytics

Where this session sits in the course

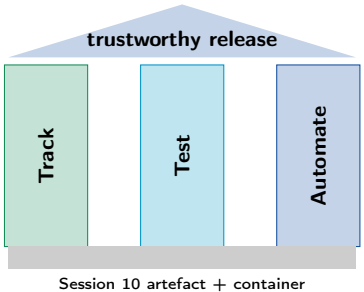


Takeaway: Session 10 made the model callable; today makes every release traceable, tested and automatic.

Learning outcomes

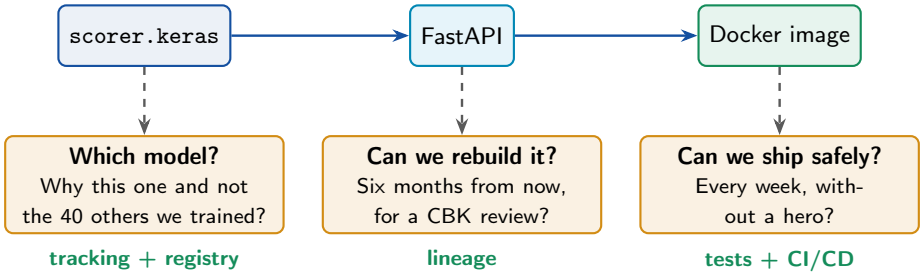
By the end of this session you will be able to:

1. **Explain** why an ML model is only reproducible if code, data, config, environment and seed are all recorded.
2. **Track** experiments and lineage with MLflow.
3. **Govern** releases through a model registry with aliases.
4. **Design** a layered testing strategy for code, data and models.
5. **Automate** test, train, build and deploy with GitHub Actions.



Where Session 10 left us

A working container, and three uncomfortable questions



Roadmap

Why reproducibility is hard

Experiment tracking with MLflow

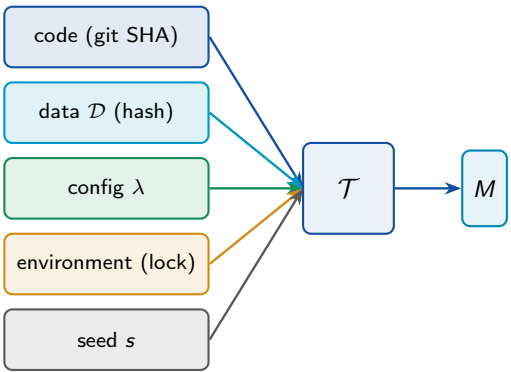
The model registry

Testing ML systems

CI/CD with GitHub Actions

A model is a function of five inputs

Record all five or you cannot rebuild the model

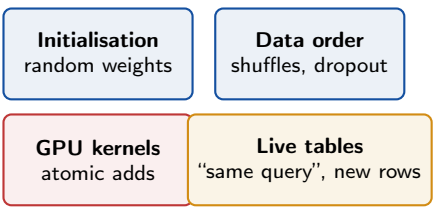


$$M = \mathcal{T}(\text{code}, \mathcal{D}, \lambda, \text{env}, s)$$

- Change any argument and you get a different M.
- Most teams record λ carefully and forget $\mathcal{D}$ and env.
- **Lineage** = a stored value for every argument.

Sources of non-determinism

Even a fixed seed is not always enough




```
import keras, tensorflow as tf

keras.utils.set_random_seed(42)    # python,
     numpy, TF

tf.config.experimental.enable_op_determinism()
```

- Determinism costs speed on GPUs; use it for audit rebuilds, not every sweep.
- Snapshot the data: a SQL query is not a dataset.

Roadmap

Why reproducibility is hard

Experiment tracking with MLflow

The model registry

Testing ML systems

CI/CD with GitHub Actions

Anatomy of a run

Everything needed to compare, and to rebuild

Run 8c02**: "dense32-l2"**

Params	Metrics	Artefacts	Tags	System
units=32 lr=3e-4 l2=1e-4	val_auc per epoch cost_kes=219 p99_ms=31	model calibration plot confusion matrix	git_sha data_sha256 owner	start, duration host, user env

- **Params**: set once. **Metrics**: time series (per step). **Artefacts**: files. **Tags**: searchable labels.

MLflow in four components

Open source, framework-agnostic, self-hostable [Zaharia et al., 2018]

Tracking
runs, params, metrics, artefacts

Models
one packaging format, many "flavours" (Keras, pyfunc)

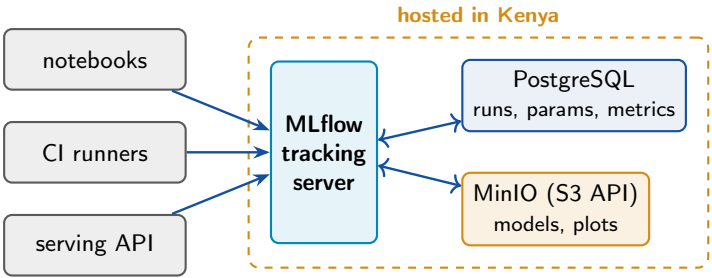
Registry
named models, versions, aliases

Tracing
spans for LLM and agent calls

today: Tracking, Models and Registry

Tracking server architecture

Metadata in SQL, files in an object store

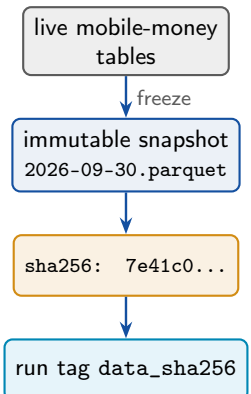


Kenyan context

Artefacts and logged samples may contain personal data. Keeping the stores in-country simplifies DPA 2019 cross-border transfer obligations (Part VI).

Version the data by hashing it

A content hash turns "the September data" into an identity




```python
import hashlib

def file_sha256(path, chunk=1 << 20):
        h = hashlib.sha256()
        with open(path, "rb") as f:
                while block := f.read(chunk):
                        h.update(block)
return h.hexdigest()

DATA_SHA = file_sha256(SNAPSHOT)
mlflow.log_input(mlflow.data.from_pandas(
train_df, source=SNAPSHOT, name="train"))
```

- DVC does the same at scale: hash, store remotely, pull by hash.

Log the model with a signature

The signature is the Session 10 contract, stored with the weights

```
from mlflow.models import infer_signature

example = {k: v[:5] for k, v in X_val.items()}
signature = infer_signature(example, model(example).numpy())

info = mlflow.keras.log_model(
        model,
        name="model",                          # MLflow 3 (artifact_path= in 2.x)
        signature=signature,
        input_example=example,
        metadata={"threshold": 0.074, "cost_fn_kes": 10000, "cost_fp_kes": 800},
        )
print(info.model_uri)                        # the URI a registry version points to
```

Key idea

Signature, input example and threshold travel together: the MLflow model is the Session 10 artefact, now tracked.

Pitfall

No signature, no schema enforcement at load time. Wrong column order then fails silently.

Roadmap

Why reproducibility is hard

Experiment tracking with MLflow

The model registry

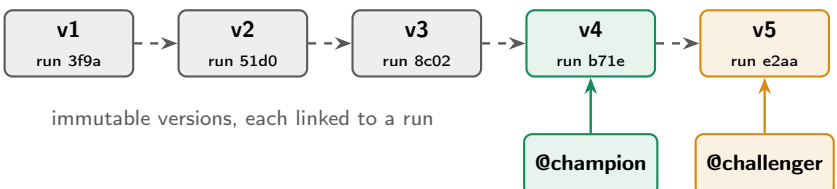
Testing ML systems

CI/CD with GitHub Actions

Registered models, versions and aliases

A registry turns a run artefact into a governed release

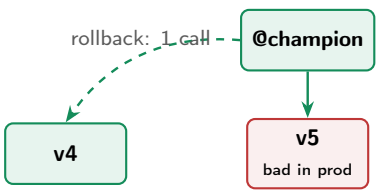
registered model: `thinfile-scorer`



- **Version**: immutable, numbered, points back to its run (lineage).
- **Alias**: a movable, named pointer. Serving code loads the alias.

Aliases, not stages

Promotion and rollback become one pointer move



- The fixed stages (Staging, Production, Archived) are deprecated in recent MLflow.
- Aliases are free-form: `@champion`, `@challenger`, `@shadow`.
- Rollback does not rebuild anything: it repoints.

Roadmap

Why reproducibility is hard

Experiment tracking with MLflow

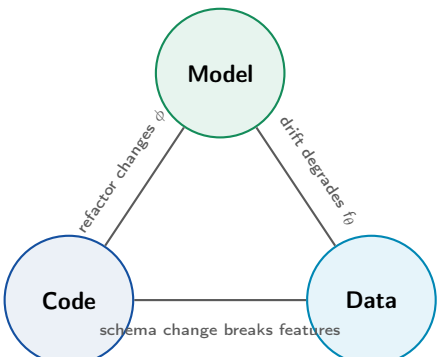
The model registry

Testing ML systems

CI/CD with GitHub Actions

Why ML testing is different

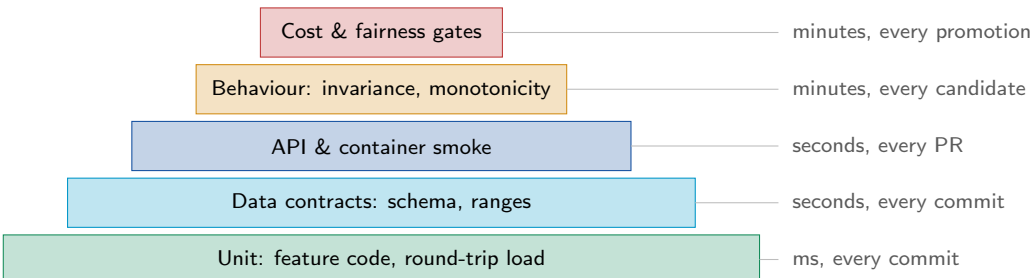
Three things can break, and only one of them is code



- Classic unit tests cover **code**.
- **Data** changes without a commit.
- **Model** "bugs" are statistical: a 2% drop in recall is not an exception.
- Google's *ML Test Score* rubric scores all three [Breck et al., 2017].

A testing pyramid for ML

Many fast cheap tests at the bottom, few slow expensive ones at the top



Takeaway: The top layer is the promotion gate: a challenger replaces the champion only if it is credibly cheaper, fair across regions and within the latency budget.

Roadmap

Why reproducibility is hard

Experiment tracking with MLflow

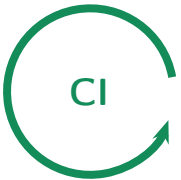
The model registry

Testing ML systems

CI/CD with GitHub Actions

CI, CD and CT

Three loops with three different triggers



trigger: pull request
lint, unit, data, API tests



trigger: merge to main
build, scan, push, deploy

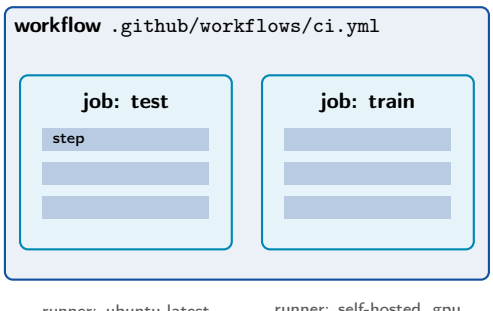


trigger: schedule or drift
retrain, gate, register

- CI and CD come from software engineering. **CT** is what makes it MLOps: the *data* can trigger a release.

GitHub Actions anatomy

Workflows contain jobs, jobs contain steps, runners execute jobs

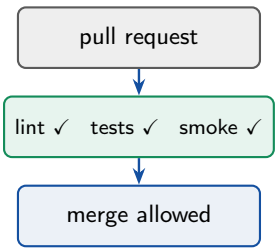


- **Triggers** (`on:`): push, pull request, schedule, manual.
- **Jobs** run in parallel unless linked by `needs:`.
- **Steps** run a shell command or a reusable action.
- **Environments** add approvals and scoped secrets.

Workflow 1: CI on every pull request

The Session 10 tests become the merge gate

```yaml
name: ci
on:
        pull_request:
branches: [main]
jobs:
test:
        runs-on: ubuntu-latest
steps:
        - uses: actions/checkout@v4
        - uses: actions/setup-python@v5
with:
        python-version: "3.11"
cache: pip
        - run: pip install -r requirements.lock
        - run: pip install -r requirements-dev.txt
        - run: ruff check . && mypy app
        - run: pytest tests/unit tests/data tests/api
        - run: docker build -t scorer:ci .
        - run: ./scripts/smoke.sh scorer:ci
```

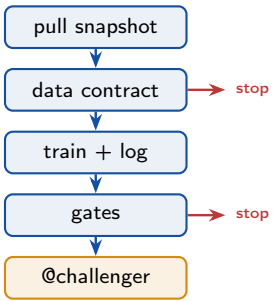


- Make the job a **required status check** in branch protection.

Workflow 2: continuous training

Retrain on schedule or on demand; register only what passes the gates

```yaml
name: train-and-validate
on:
        workflow_dispatch:
schedule:
        - cron: "0 2 * * 1"    # Mon 05:00 EAT
jobs:
        train:
                runs-on: [self-hosted, gpu]
env:
        MLFLOW_TRACKING_URI: ${{ vars.MLFLOW_URI }}
steps:
        - uses: actions/checkout@v4
        - run: dvc pull data/snapshots/latest
        - run: pytest tests/data
        - run: python -m pipeline.train
        - run: pytest tests/model  # gates
        - run: python -m pipeline.register
```



Workflow 3: CD, build and scan

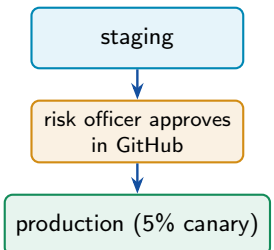
Nothing reaches the registry without passing a vulnerability scan

```yaml
name: cd
on:
push:
branches: [main]
env:
        IMAGE: ghcr.io/ilabafrica/thinfile-scorer:${{ github.sha }}
jobs:
build:
        runs-on: ubuntu-latest
permissions: {contents: read, packages: write}
steps:
        - uses: actions/checkout@v4
        - run: python -m pipeline.fetch_model --alias champion
        - run: docker build -t "$IMAGE" .
        - run: ./scripts/trivy_scan.sh "$IMAGE"     # exit 1 on CRITICAL CVEs
        - uses: docker/login-action@v3
with:
        registry: ghcr.io
        username: ${{ github.actor }}
        password: ${{ secrets.GITHUB_TOKEN }}
        - run: docker push "$IMAGE"
```

Workflow 3: CD, deploy through environments

A protected environment is where the human sign-off lives

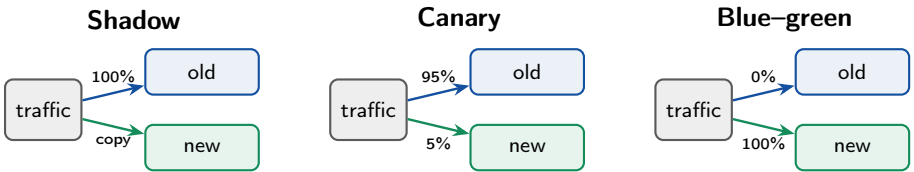
```
deploy-staging:
needs: build
runs-on: ubuntu-latest
environment: staging
steps:
        - run: ./deploy.sh staging "$IMAGE"
        - run: ./scripts/smoke.sh https://staging.
    scorer
deploy-production:
needs: deploy-staging
runs-on: [self-hosted, nairobi-dc]
environment: production      # required reviewers
steps:
        - run: ./deploy.sh production "$IMAGE" --
    canary 5
```



- Production secrets exist only in the `production` environment.

Deployment strategies

Credit labels arrive weeks late, so choose accordingly

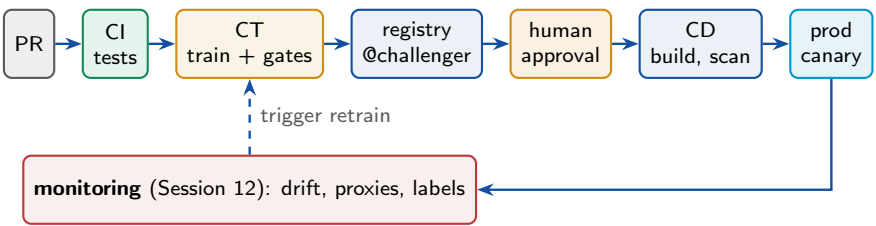


Key idea

Defaults are observed 30+ days after disbursement. Canary on *proxies* (approval rate, score distribution, latency, errors); shadow first to compare decisions without risk.

The whole pipeline, end to end

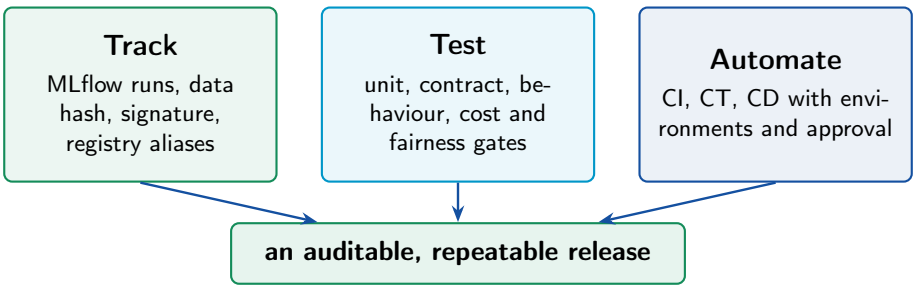
From a pull request to a monitored production model



- Every arrow leaves a record: a commit, a run, a version, an approval, an image digest.

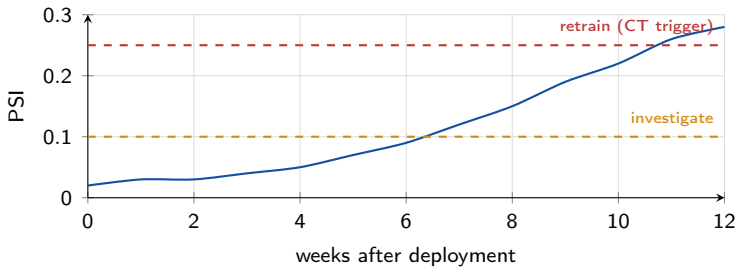
Summary

Session in one picture: Track, test, automate



Next session: the model meets the world

Session 12, Monday 19 October: monitoring, drift and system design



- Today's CT workflow gets its trigger next week: drift detection with Evidently.
- **A4 is due today**; capstone systems (Weeks 14–15) must use this pipeline.

References

Chapter 10 of the course book develops every slide in prose

- **Huyen** (2022), *Designing Machine Learning Systems*, Ch. 6 (experiment tracking and versioning) and Ch. 9 (continual learning, test in production).
- **Breck et al.** (2017), "The ML Test Score: a rubric for ML production readiness", IEEE Big Data.
- **Ribeiro et al.** (2020), "Beyond accuracy: behavioral testing of NLP models with CheckList", ACL.
- **Zaharia et al.** (2018), "Accelerating the machine learning lifecycle with MLflow", IEEE Data Eng. Bull.
- **Kreuzberger et al.** (2023), "Machine Learning Operations (MLOps): overview, definition, and architecture", IEEE Access.
- **Documentation**: MLflow (tracking, model registry and aliases); GitHub Actions (workflows, environments); DVC.